

## # Loop Statements:

- Loops are the control flow statements which helps the programmer to execute some set of instructions repeatedly for desired number of iterations (cycles) based on a condition.

- Loops make our code more readable and improves performance by eliminating the repetition of a code.

- ~~the~~ +

- We have 4 types of loop statements in java.

(i) for loop

(ii) while loop

(iii) do while loop

(iv) for each loop / enhanced / advanced for loop  
(1.5 version)

- Loop has 3 main parts:

(a) initialization statement

(b) condition statement

(c) updation statement

### (A) Initialization Statements:

- The initialization statements is the starting point of the loop where we declare a variable.

- It is used to initialize loop control variable (counter-variable).

- The initialization part of loop gets executed only once when loop starts.

- We can also declare multiple variables at initialization, (separated using comma)

```

Ex: class loopEx
{
    public static void main()
    {
        int end = 5; initialization
        for (int start = 1; start <= end; start++)
        {
            SOP("hello" + start);
        }
    }
}

```

### (B) Condition statement.

- The condition statement controls the execution of a loop.
- It must be executed before every loop iteration.
- A condition must be a boolean expression.
- If condition is true loop body gets execute.
- If condition is false loop gets terminate (stop).
- We can use any operator but it must return boolean value.

```

Ex: class LoopEx1
{
    public static void main()
    {
        int start = 1;
        int end = 5;
        while (start <= end)
        {
            SOP("Hello" + start);
            start++;
        }
    }
}

```

### (c) Updation Statement

- This statement gets executed after each iteration of loop body.
- It helps the counter variable to get updated (reinitialization).
- Updation statement helps the condition to become false.
- We can use multiple statements in updations.
- If the counter variable is not updated then the same condition remains same and loops runs infinitely.

Ex: class LoopEx2

```
{ public static void main()  
  { int start=1; //initialization (start point)  
    int end=5; //end point  
    do {  
      SOP("Hello" + start);  
      start++; //updation  
    } while (start <= end); //condition  
  }  
}
```

### [i] For loop

- for is a looping statement which helps the programmer to execute some set of instructions repeatedly for desired no. of times based on a condition.
- we use for loop when we know the no. of iterations.

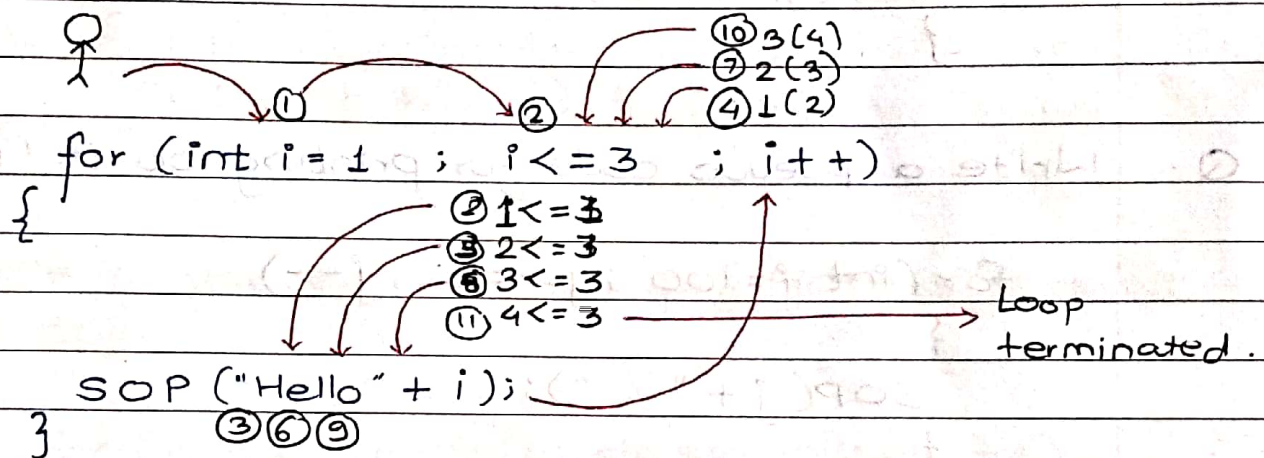
## Syntax:

```

keyword → for (initialization ; condition ; updation)
{
  Loop body → // statements
}

```

## # Execution flow for "for" loop



Output: Hello 1

Hello 2

Hello 3

- In for loop, initialization, condition, updation are declared at the same line.
- First initialization takes place where the counter variable gets initialized with a value.
- Then the control checks the condition and executes loop body if the condition is true. or else loop gets terminated if the condition is false.
- After completion of an iteration then the control executed updation statement and again condition will be executed.

- The loop will continue iterations ~~at~~ until condition becomes false.

Q. Write a pseudo code for printing 1 to 100 numbers

```
for (int i = 1 ; i <= 100 ; i++)  
{  
    SOP(i + " ");  
}
```

Q. Write a pseudo code for printing 100 to 1 numbers

```
for (int j = 100 ; j >= 1 ; j--)  
{  
    SOP(j + " ");  
}
```

Q. Write a pseudo code for printing even numbers between 1 to 100

```
for (int k = 1 ; k <= 100 ; k++)  
{  
    if (k % 2 == 0)  
        SOP(k + " ");  
}
```

Q. Write a pseudo code for printing odd numbers between 1 to 100

```
for (int l = 1 ; l <= 100 ; l++)  
{
```

```

if (l % 2 != 0)
    SOP(k +
        SOP(l + " ");
}

```

Q Write a pseudo code for printing alphabets A to Z

```

for (char ch = 'A' ; ch <= 'Z' ; ch++)
{
    SOP(ch + " ");
}

```

Q Write a pseudo code for printing alphabets a to z

```

for (char ch = 97 ; ch <= 122 ; ch++)
{
    SOP(ch + " ");
}

```

Q. Write a pseudo code for printing digits from 0 to 9

```

for (int p =
    for (char p = 48 ; p <= 57 ; p++)
    {
        SOP(p + " ");
    }
}

```

Q. Write a pseudo code for printing 'A' to 'Z', 'a' to 'z' and '0' to '9'.

```
for (char n = 1; n <= 127; n++)
{
    if (n >= 65 && n <= 90 || n >= 97 && n <= 122
        || n >= 48 && n <= 57)
        SOP(n + " ");
}
```

Q. Write a pseudo code for printing only special characters.

```
for (char ch = 1; ch <= 127; ch++)
{
    if (! (ch >= 65 && ch <= 90 || ch >= 97 && ch <= 122
        || ch >= 48 && ch <= 57))
        SOP(ch + " ");
}
```

Q. Write a pseudo code for printing alphabets lowercase 'z' to 'a'.

```
for (char ch = 'z'; ch >= 'a'; ch--)
{
    SOP(ch + " ");
}
```

Q. Write a pseudo code for printing 'A' to 'z' with their ascii values.

```
for (char ch = 65; ch <= 90; ch++)
{
    SOP(ch + ":" + (ch + 0) + " ");
}
```

Q. Write a pseudo code for print a to z with their ascii value

```
for (char ch = 97; ch <= 122; ch++)
{
    SOP(ch + ':' + (ch + 1) + " ");
}
```

Q. Write a pseudo code to print a to z alternately (A, c, E, G, ...).

```
for (char ch = 'A'; ch <= 'Z'; ch += 2)
{
    SOP(ch + " ");
}
```

Q. Write a pseudo code to print this pattern (A, c, F, J, O, U)

```
int step = 1;
for (char ch = 'A'; ch <= 'Z'; ch += step)
{
    SOP(ch + " ");
    step++;
}
```

Q. Write a pseudo code for find vowels from upper case and lowercase using arrow operator.

```
for(ch = 65 ; ch <= 122 ; ch++)
{
    if (ch >= 65 && ch <= 90 || ch >= 97 && ch <= 122)
    {
        String op = switch (ch)
        {
            case 'a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U':
                → "VOWELS";
            default → "CONSONANTS";
        };
        SOP(ch + ":" + op);
    }
}
```

Ex: class LoopEx

```
{
    psu main()
    {
        SOP("Starts");
        Loop declaration → for (int i = 1 ; i <= 5 ; i++);
        Loop block {
            {
                SOP(i);
            }
            SOP("ends");
        }
    }
}
```

NOTE: A block declared are loop declaration will be associated with it.

- When we declare a semicolon or any statement between them, it breaks their association.

```

{ for();
  SOP("Hello"); // No CTE
}

```

Page No.

Date

- It will create a CTE only if the members such as variables is accessed outside its scope

Ex:

```

for(int i = 1; i <= 5; i++)
  SOP(i);

```

- Blocks are optional and only one statement is considered as a part of loop, if we omit them.

```

Ex: for(int i = 1, j = 1; i <= 10; i++, j++)
    { SOP(i + ":" + j);
    }

```

Output: 1:1

2:2

3:3

:

10:10

- we can declare multiple variables in initialization statement, but the variables will be of same type.

- we can also declare multiple updation statements at once using commas.

```

Ex: for(int i = 1; i <= 10)      for(int i = 1; i <= 10)
    { i++;                      { i++;
    SOP(i + " ");              SOP(i + " ");
    }                          }

```

OP: 2 4 6 8 10

OP:

- If we want, we can declare the updation statements inside the loop body.

- If we are not providing any updation statements there is a chance of loop going into infinity.

```
for(byte i=1; ; )
{ SOP(++i + " ");
}
```

- When a programmer fails to provide or add a condition then compiler provides a default condition i.e. true

- If a loop runs for infinite times and there is no break point to terminate it then the statements after loop becomes "unreachable" which creates CTE i.e. "unreachable statements."

Ex :

```
SOP("starts");
for(int i=1; ; )
{ SOP(i);
}
SOP("ends");
```

OP: CTE 'unreachable statements'.

Ex: boolean cndt = true;

```
SOP("starts");
```

```
for(byte i=1; cndt; )
```

```
{ SOP(++i + " ");
```

```
}
```

```
SOP("ends");
```

OP: 1 2 3 4 ... 127 - 128 .....

-1 0 1 2 ... infinite times

- When the condition is constant i.e. true the statements after loop becomes unreachable.

- When we specify the condition inside a non-final variable then it doesn't create unreachable statements because the condition might change.

```
Ex: final boolean CNDT = true;
    SOP("start");
    for (byte i = 1; CNDT;)
    { SOP(++i + " ");
    }
    SOP("ends");
```

Op: SOP(end) ← CTE "unreachable statements."

```
Ex: SOP("starts");
    for (byte i = 1; false;)
    {
        SOP(++i + " "); // CTE "unreachable st"
    }
    SOP("ends");
```

- When the constant condition is false the loop cannot perform any iteration and it leads to a CTE i.e. unreachable statements.

- Because, control can never execute loop body.

```
Ex: boolean cndt = false;
    SOP("starts");
    for (byte i = 1; cndt;)
    { SOP(++i + " ");
    }
    SOP("ends");
```

Op: starts  
ends

- The `cond` is a non final variable which might change due to, which it doesn't create CTE.

Ex: `SOP("starts");` O/p: starts  
`for (byte i = 1; true;)` 2 3 4 5 6 7 8 9  
`{ SOP(++i + " ");` 10 11 12 13 14 15  
`- if (i == 15) break;` ends  
`}`  
`SOP("ends");`

- In the above example, constant `cond == true` creates an infinite loop and there is a `break` statement to terminate this loop, so that's why it doesn't create CTE. 'unreachable statements'.

Ex: `SOP("starts");`  
`int b = 5;`  
`for (int i = 1; i <= b; i++, b--)`  
`{`  
`SOP("hello");`  
`}`  
`SOP("ends");`

O/p: starts

hello  
 hello  
 hello  
 hello  
 hello

ends

Ex: `SOP("starts");`  
`int b = 5;`  
`for (int i = 1; i <= b; i++, b--)`  
`{ SOP("hello");`  
`}`

```
SOP("end's");
```

O/P: starts

hello

hello

hello

ends.

```
Ex: SOP("starts");
```

```
final int b = 5;
```

```
for (final int i = 1; i <= b; i++)
```

```
{
```

```
    SOP("hello");
```

```
    if (i == b) break;
```

```
}
```

O/P: infinite

hello

```
System.out.println("ends");
```

Due to break statement, control thinks that sometime the loop will break and no CTF is displayed.

when loop runs for infinite times with constant condition and if there is a break statement in it. It won't create any CTF.

```
Ex: SOP("starts");
```

```
int b = 5;
```

```
for (int i = 1; i <= b; i++, b--)
```

```
{
```

```
    if (i == b) break;
```

```
    SOP("hello");
```

```
}
```

```
SOP("end");
```

O/P: starts

hello

hello

ends

- Initialization of counter variable can be done outside the loop (before loop starts) or inside loop declaration statement

```
Ex : int i = 1; ← initialization
      for ( ; i <= 5 ; i++) ← loop declaration
      { SOP("hello");
      }
```

- when we declare a counter variable outside the loop we need to ensure that same variable name in the same scope should not be used again or else it creates CTE.

- Updation statement can be declared inside the loop body or loop declaration

```
Ex: int i = 1;
     for ( ; i <= 5 ; )
     {
       SOP("hello");
       i++;
     }
```

- when initialization and updation is more complex we declare them separately.

- If we want to declare them separately we should use "while" loop.

## #While Loop

- while is a looping statement which helps the programmer to execute set of instructions repeatedly until the condition becomes false

- we use while loop when we don't know the no. of iterations.

### Syntax:

initialization;

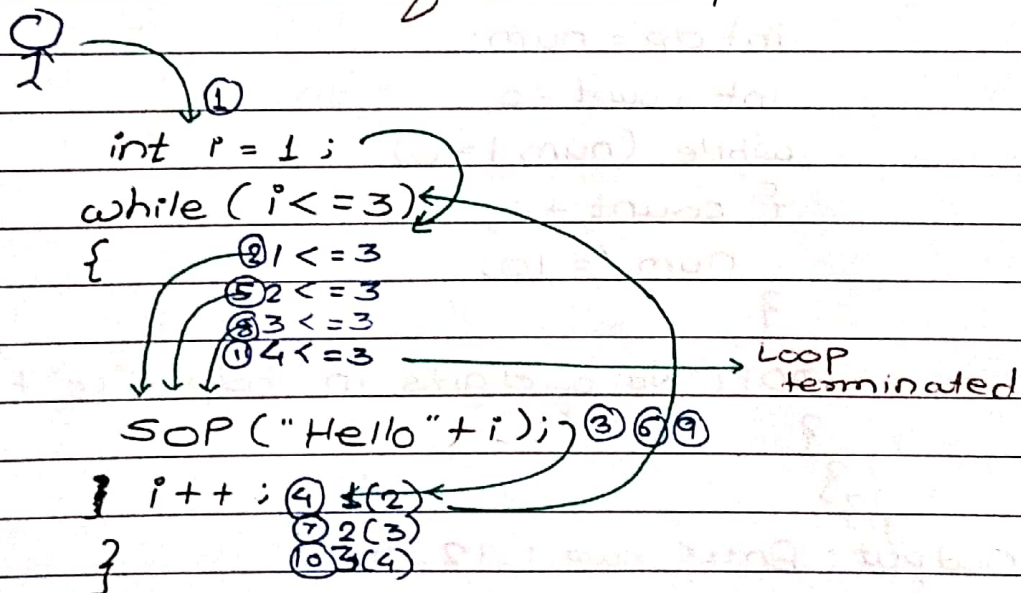
keyword  $\rightarrow$  while (condition)  
{

loop body  $\rightarrow$  [ // statements  
                  update;  
                  }

$\rightarrow$  condition is in boolean format that is either true or false

$\rightarrow$  update can be any equation format. commonly used is increment and decrement.

### #Execution Flow of while Loop:



Output: Hello 1

Hello 2

Hello 3

- Control first goes to initialization and inheritance initializes the value in memory
- After initialization, control goes to check the condition
- If the condition is true the control goes to check the condition inside the while loop, execute the statements and update the value in memory. This keeps on executing in loop format until loop gets break.
- If the condition is false the loop gets terminated. In other words, loop gets break.

Ex: import java.util.Scanner;

class DigitCount

{

public static void main(String [] args)

{ Scanner sc = new Scanner(System.in);

SOP("Enter num");

int num = sc.nextInt();

int dp = num;

int count = 0;

while (num != 0)

{ count ++;

num /= 10;

}

SOP("No of digits in " + dp + " is " + count);

}

}

}

Output: Enter num : 1234

No of digits in 1234 is 4

\* When to use while loop:

- when we don't know the number of iterations we should use while loop.
- when we want to check the condition before entering the loop is necessary.
- when the initializing or initialization and updation is complex.

NOTE: - For loop and while loop are different in terms of syntax.

- After compilation them into bytecode, the loading instructions for them is same.
- There is no significant speed difference between for and while loop in java.
- In most cases, both performs equally fast. because, java compiles both loops into similar bytecode.

Ex: class LoopEx

{

public static void main()

{

for (int i = 1; i <= 5; i++)

{

}

int r = 1;

while (r <= 5)

{

i++;

}

}

}

# bytecode instructions:

```
class LoopEx {
```

```
    LoopEx();
```

```
    public static void main(java.lang.String[]);
```

```
    code :
```

```
    0 : iconst-1 → push 1 (i=1)
```

```
    1 : istore-1 → store in local variable i
```

```
    2 : iload-1 → load i
```

```
    3 : iconst_5 → push 5
```

```
    4 : if-icmplt 13
```

→ if i value is <sup>greater</sup> less than 5 exit loop  
and <sup>start</sup> execute from line 13

```
    5: iinc 1, 1 → i++ (increment value of i by 1)
```

```
    10: goto 2
```

→ jump back to line 2 to check condition

```
    13: return → exit method
```

Same instructions will be generated for while loop

Q. Write a pseudo code to print 1 to 100 (while loop)

```
int i = 1;
```

```
while (i <= 100)
```

```
{
```

```
    sop(i + " ");
```

```
    i++;
```

```
}
```

Q. Write a pseudo code to print 100 to 1 (while loop)

```
int k = 100;
```

```
while (k >= 100)
```

Page No.  
 Date

```

{
    SOP(k + " ");
    k--;
}
  
```

Q. Write a pseudo code to print even numbers from 1 to 100 using while loop

```

int j = 1;
while (j <= 100)
{
    if (j % 2 == 0)
        SOP(j + " ");
    j++;
}
  
```

Q. Write a pseudo code to print odd numbers from 1 to 100 using while loop.

```

int k = 1;
while (k <= 100)
{
    if (k % 2 != 0)
        SOP(k + " ");
    k++;
}
  
```

Q. Write a pseudo code to print 'A' to 'Z' (while loop)

```

char ch = 65;
while (ch <= 90)
{
  
```

```
SOP(ch + " ");
ch++;
}
```

Q. Write a pseudo code to print 'a' to 'z' using while loop.

```
ch1 = 97;
while (ch1 <= 122)
{
    SOP(ch1 + " "); ch1++;
}
```

Q. Write a pseudo code to print 0 to 9 digits

```
ch2 = 48;
while (ch2 <= 56)
{
    SOP(ch2 + " ");
    ch2++;
}
```

Q. Write a pseudo code to print all 'A' to 'z', 'a' to 'z' and '0' to '9'.

```
char m = 'A';
while (m <= 127)
{
    if (m >= 65 && m <= 90 || m >= 97 && m <= 122
        || m >= 48 && m <= 57)
        SOP(m + " ");
    m++;
}
```

Q. Write a pseudo code to print only special characters

```

char w = 1;
while (m <= 127)
{
    if (! (w >= 65 && w <= 90 || w >= 97 && w <= 122
        || w >= 48 && w <= 57))
        SOP(w + " ");
    w++;
}

```

Q. Write a pseudo code to print 'z' to 'a' (while loop)

```

char z = 'z';
while (z >= 'a')
{
    SOP(z + " ");
    z--;
}

```

Q. Write a pseudo code to print characters with their ascii value (A to Z)

```

char chl = 65;
while (chl <= 90)
{
    SOP(chl + " : " + (chl + 0) + " ");
    chl++;
}

```

Q. Write a pseudo code to print characters with their ascii value (a to z)

```

char ch2 = 97;
while (ch2 <= 122)
{
    SOP(ch2 + " : " + (ch2 * 1) + " ");
    ch2++;
}

```

Q. Write a pseudo code to print alternate alphabets

```

char ch3 = 'A';
while (ch3 <= 'Z')
{
    SOP(ch3 + " ");
    ch3 += 2;
}

```

Q. Write a pseudo code to print this alphabet pattern  
(A, C, E, J, O, U)

```

int step = 1;
char ch = 'A';
while (ch <= 'Z')
{
    SOP(ch + " ");
    step++;
    ch += step;
}

```

Q. Write a pseudo code to find vowels from uppercase and lowercase alphabets using arrow operator in switch

```

char ch = 65;
while (ch <= 122)
{
    if (ch >= 65 && ch <= 90 || ch >= 97 && ch <= 122)
    {
        string op = switch (ch) {
            case 'a', 'e', 'i', 'o', 'u',
                  'A', 'E', 'I', 'O', 'U' -> "VOWEL";
            default -> "CONSONANT";
        };
        SOP(ch + ": " + op + " ");
        ch++;
    }
}

```

Ex: final int a = 1;

SOP("starts");

while (a <= 5) {

SOP(a);

}

SOP("ends");

O/P: CTF

unreachable  
statements.

- When there is a constant condition i.e true the statement after loop becomes unreachable

Ex: final int a = 1;

SOP("starts");

while (a <= 5) {

SOP(a);

if (true) break;

}

SOP("ends")

O/P: starts

ends.

- The break statement can terminate the infinite loop that's why it doesn't create CTE.

## Difference between for and while loop

| For Loop                                                                                                         | while loop                                                                                                   |
|------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| - for loop is used when we know the number of iterations                                                         | - while loop is used when we don't know the number of iterations                                             |
| - In for loop, Initialization, Updation and Condition are declared at same line.                                 | - In while loop, Initialization, Updation and Condition are declared at <sup>different lines</sup> same time |
| - When initialization and updation simple we use for loop.                                                       | - when initialization and updation is complex we use while loop.                                             |
| - In for loop, if programmer fails to provide a condition, the compiler provides a default condition i.e. "TRUE" | - In while loop, if programmer fails to provide a condition, it creates CTE                                  |
| - The counter variable can be declared before loop starts or in loop declaration                                 | - The counter variable must be declared before loop starts                                                   |
| - Syntax:<br><pre>for (ini; cnd; upd) {     // statements }</pre>                                                | - Syntax:<br><pre>inp; while (cnd) {     // statements     upd; }</pre>                                      |

## # do while Loop

- do while is a looping statement which helps the programmer in executing some set of instructions repeatedly.

- we use do-while loop, when we don't know the ~~next~~ no. of iterations and task is more important than the condition.

Syntax:

```

      initialization ;
      keyword → do {
                  // statements
                  updation ;
                  }
               while (condition);
  
```

## # Control flow of do-while Loop [Execution]:

- first, we initialize the counter variable i.e <sup>starting</sup> ~~starting~~ point of loop.

- Then, control executes do block, performs task along with updation of counter variable.

- This completes one iteration of the loop, and then control checks for the condition in while.

- If condition evaluates to true then the control starts the execution of next iteration by executing do block again.

- If the condition is false, loop gets terminated.

# Byte code instruction for do-while loop.

```
class LoopExample
{
```

```
    private LoopExample() {
    }
```

```
    public static void main(String [] args)
    {
```

```
        int i = 1;
```

```
        do {
```

```
            i++;
```

```
        }
```

```
        while(i <= 5);
```

```
    }
```

```
}
```

```
}
```

```
class LoopExample {
```

code:

0: iconst-1 → push constant 1 (initialized i = 1)

1: istore-1 → store value into local variable.

2: iinc 1, 1 → Increment local variable 1 by 1 (i++)

5: iload-1 → Load i from local variable

6: iconst-5 → push constant 5

7: if-icmple 2 → If i is less than equal to 2  
then jump to instructions no. 2

10: return → end of a method.

```

Ex: int i = 1;
    do {
        SOP(i);
        i++;
    } while (i >= 10);

```

OP: 1

- do-while is also known as exit control loop. because, the condition is checked at the end of a loop.

- Due to which, do-while performs at least 1 iteration even if the condition is false.

- In do-while loop, blocks are optional.

```

Ex: int i = 1;
    do {
        SOP(i++);
    } while (i <= 5);

```

```

Ex: int i = 1;
    do {
        SOP(i);
    } while (i++ <= 5);

```

```

Ex: SOP("starts");

```

OP: CTE

```

final int i = 1;

```

unreachable

```

do {

```

statements.

```

    SOP(i);

```

```

} while (i <= 5);

```

```

    SOP("ends");

```

```

Ex: SOP("starts");

```

OP: starts

```

final int i = 1;

```

1

```

do {

```

ends

```

    SoP(i);
} while(i >= 5);
    SoP("ends");

```

NOTE: Semi colon is a part of do-while loop which must be placed at the end of while i.e condition.

### Difference between while loop and do while loop

| while loop                                                                | do-while loop                                                                                                     |
|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| - while loop is used when task condition is more important than the task. | - do-while loop is used when task is more important than condition.                                               |
| - In while, condition is checked at the start (entry)                     | - In do-while condition is checked at the end (exit)                                                              |
| - while loop is also known as entry control loop                          | - do-while is also known as exit control loop.                                                                    |
| - In while loop, semicolon is not a part of syntax                        | - In do-while semicolon is a part of syntax.                                                                      |
| - If condition is false                                                   | - If the condition is false still, loop iterates once.                                                            |
| - Initialization of counter variable must be done before loop starts      | - Initialization of counter variable can be done before loop starts or inside do block (It creates infinite loop) |

Syntax:

```
ini ;
while(cnd)
{
  //statements
  upd;
}
```

Syntax;

```
ini ;
do {
  //statements;
  upd;
} while (cnd);
```

Ex: SOP("starts");

```
int i;
```

```
do {
```

```
  i = 1;
```

```
  SOP(i++);
```

```
}
```

```
while (i <= 1);
```

```
SOP("ends");
```

OP: starts

1

ends

- The counter variable  $i$  can be initialized before loop starts or inside loop body.
- Based on the condition, it might create infinite loop.

Q. Write a pseudo code to print 1 to 100 (do-while loop)

```
int j=1;
do {
    SOP(j + " ");
    j++;
} while(j <= 100);
```

Q. Write a pseudo code to print 100 to 1 (do-while loop)

```
int k=100;
do {
    SOP(k + " ");
    k--;
} while(k >= 1);
```

Q. Write a pseudo code to print even numbers bet<sup>n</sup> 1 to 100 using do-while loop

```
int e=1;
do {
    if (e%2 == 0)
        SOP(e + " ");
    e += 2;
} while(e <= 100);
```

Q. Write a pseudo code to print odd numbers

```
int o=1;
do {
    if (o%2 != 0)
        SOP(o + " ");
    o++;
} while(o <= 100);
```

Q. Write a pseudo code to print A to Z

```
char ch = 'A';
do {
    SOP(ch + " ");
    ch++;
} while (ch <= 'Z');
```

Q. Write a pseudo code to print a to z

```
char ch1 = 97;
do {
    SOP(ch1 + " ");
    ch1++;
} while (ch1 <= 122);
```

Q. Write a pseudo code to print 0 to 9

```
char ch2 = 48;
do {
    SOP(ch2 + " ");
    ch2++;
} while (ch2 <= 57);
```

Q. Write a pseudo code to print all 'A' to 'Z' & 'a' to 'z' and '0' to '9'

```
char ch3 = 1;
do {
    if (ch3 >= 48 && ch3 <= 57 || ch3 >= 65 && ch3 <= 90 ||
        ch3 >= 97 && ch3 <= 122)
        SOP(ch3 + " ");
    ch3++;
} while (ch3 <= 127);
```

Q. Write a pseudo code to print only special characters

```

char ch4 = 1;
do {
    if (!(ch4 >= 48 && ch4 <= 57 || ch4 >= 65 &&
        ch4 <= 90 || ch4 >= 97 && ch4 <= 122))
        SOP(ch4 + " ");
    ch4++;
} while (ch4 <= 127);

```

Q. write a pseudo code to print 'z' to 'a'

```

char ch5 = 'z';
do {
    SOP(ch5 + " ");
    ch5--;
} while (ch5 >= 'a');

```

Q. Write a pseudo code to print 'A' to 'z' with ascii values using do-while loop.

```

char ch6 = 'A';
do {
    SOP(ch6 + " : " + (ch6 + 0) + " ");
    ch6++;
} while (ch6 <= 'z');

```

Q. Write a pseudo code to print 'a' to 'z' with their ascii value using do-while loop

```

char ch7 = 'a';
do {

```

```

    SOP(ch7 + ":" + (ch * 1) + " ");
    ch7++;

```

```

} while (ch7 <= 122);

```

Q. Write a pseudo code to print alternate alphabet

```

char ch8 = 'A';

```

```

do {

```

```

    SOP(ch8 + " ");

```

```

    ch8 += 2;

```

```

} while (ch8 <= 'z');

```

Q. Write a pseudo code to print alphabet pattern  
[A C F I O U]

```

int step = 1;

```

```

char a = 'A';

```

```

do {

```

```

    SOP(a + " ");

```

```

    step++;

```

```

    ch += step;

```

```

} while (a <= 'z');

```

Q. Write a pseudo code to print vowels or consonants for 'A' to 'z' and 'a' to 'z'.

```

char ch = 65;

```

```

do {

```

```

    if (ch >= 65 && ch <= 90 ||

```

```

        ch >= 97 && ch <= 122)

```

```

    {

```

```

string op=" switch (ch)
{
    case 'a','e','i','o','u',
        'A','E','I','O','U' -> "VOWEL";
    default -> "CONSONANT";
}
SOP(ch + ":" + op + " ");
ch++;
} while (ch <= 122);

```

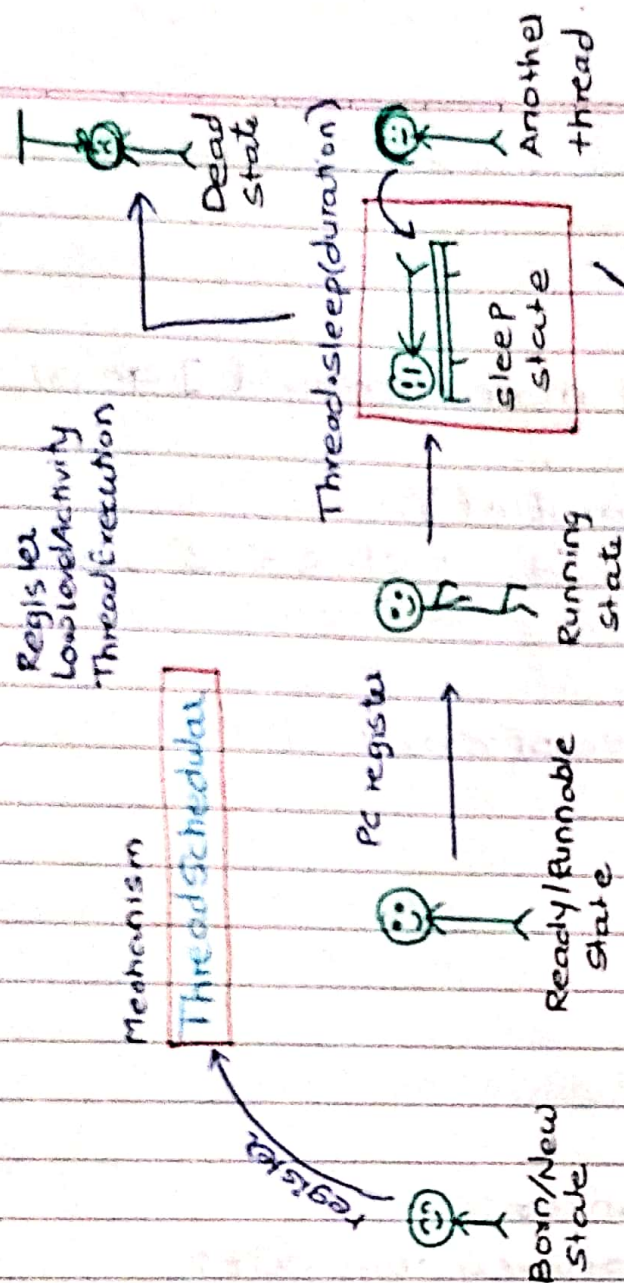
Prog:

```

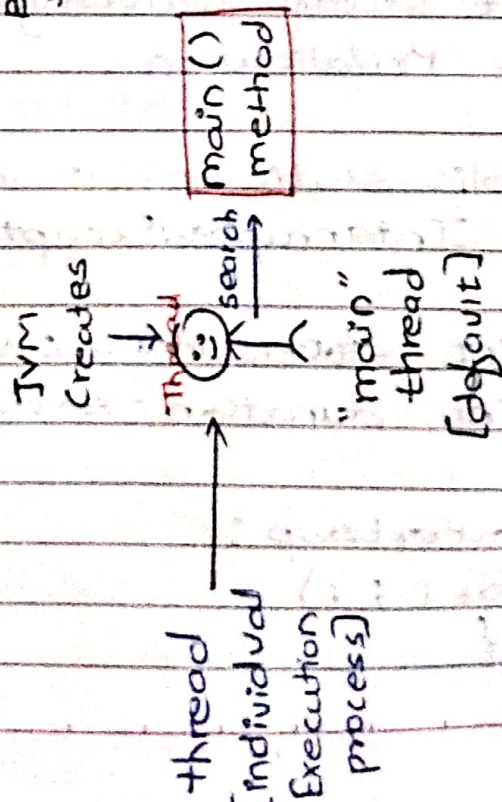
class Demo
{
    public static void main(String [] args)
        throws InterruptedException
    {
        SOP();
        for (char ch = 'A'; ch <= 'Z'; ch++)
        {
            SOP(ch + " ");
            Thread.sleep(200);
        }
        SOP();
    }
}

```

# Thread Life cycle



2 Types of thread  
 ① Daemon  
 ② NonDaemon



Q. WATP for thread sleep method

```
class ThreadDemo {
    public static void main(String [] args)
    {
        System.out.println();
        for (char ch = 'A' ; ch <= 'Z' ; ch++)
        {
            SOP(ch);
            Thread.sleep(200);
        }
        SOP();
    }
}
```

Q. WATP for pin Validation

```
import java.util.Scanner;
import static java.lang.System.out;
class PinValidation
{
    public static void main(String [] args) throws
        InterruptedException
    {
        final int STORED_PIN = 1234;
        int duration = 5000;

        outerLoop:
        for ( ; ; )
        {
```

```
int attempt = 3;
do {
    out.println();
    out.println("Enter your pin:");
    int userPin = new java.util.Scanner(System.in)
        .nextInt();
    if (STORED_PIN == userPin) {
        out.println("Phone unlocked");
        break outerLoop;
    }
    else {
        out.println("Wrong Pin");
        out.println("Attempt left " + (attempt - 1));
    }
    attempt--;
}
while (attempt != 0);

SOP();
SOP("Phone is Disabled for " + (duration/1000)
    + "secs");
Thread.sleep(duration);
duration *= 2;
}
}
}
```

## \* Break

- Break is a keyword and a branching statement in java. which is used to exit a switch statement or looping statements.

- If we write "break" outside switch or loop it creates a CTE.

## # Break in Loops:

- we can use break statement in all types of loops i.e for/while/dowhile/foreach

- If we want to terminate (exit) a loop before the loop condition evaluates to false we can use break.

```
Ex: class BreakEx
{
    main()
    {
        int sum=0;
        for (int i=1; i<=10; i++)
        {
            sum += (int)(Math.random() * 100);
            if (sum > 500) break;
        }
        SOP(sum);
    }
}
```

- A break statement must be inside a conditional statement or it must be at the end of a loop.

```

Ex: class BreakEx
{
    main()
    {
        int sum = 0;
        for (int i = 1; i <= 10; i++)
        {
            sum += (int)(Math.random() * 100);
            break;
        }
        SOP(sum);
    }
}

```

- Loop will iterate only once and some random value will be generated.

```

Ex: class BreakEx
{
    main()
    {
        for (int i = 1; i <= 10; i++)
        {
            break;
            SOP(i);
        }
    }
}

```

- CTE - unreachable statements.

because, break will terminate the execution of loop and statements after it will become unreachable. because, break is specified directly inside the loop.

# Printing hello java  $n^{\text{th}}$  times without semicolon using for.

```

class LoopEx
{
    main()
    {

```

```
for(int i=0; i++<10; SOP("Hello java"))
{
}
}
```

### \* continue

- continue is a keyword and a branching statement in java. which is used to skip the current iteration it moves the control to the next iteration.

- It doesn't terminate the loop, it jumps back to the loop's condition check (in case of for it will jump to update expression)

```
Ex: class ContinueExample
{
    main()
    {
        for(int i=1; i<=5; i++){
            SOP("starts : "+i);
            if(i%2 != 0) continue;
            SOP("ends : "+i);
        }
    }
}
```

OP: starts : 1

starts : 2

ends : 2

starts : 3

starts : 4

ends : 4

starts : 5

IMP

Ex: class ContinueEx

OP: Starts

{ main()

1

{ SOP("starts");

2

int i=1;

3

while (i!=5)

- infinite

{

if (i == 4) continue;

SOP(i);

i++;

}

SOP("ends");

{

}

\* Execution flow for continue.

- Loop starts

- condition is checked

- inside loop body

(a) if continue : is encountered, it skips the remaining statements

(b) control jumps to

(i) next condition is checked in case of while and do-while.

(ii) in case of for updation gets updation.

Ex: class ContinueEx

{ main()

{ for (int i=1; i&lt;=3; i++)

{ for (int j=1; j&lt;=3; j++) {

if (j==2) continue;

SOP("i" + i + " : j" + j);

}

}

OP: i1 : j1  
 i1 : j3  
 i2 : j1  
 i2 : j3  
 i3 : j1  
 i3 : j2

```
Ex: class BreakEx
{
    main()
    {
        for (int i = 1; i <= 3; i++)
        {
            for (int j = 1; j <= 3; j++)
            {
                if (j == 2) break;
                SOP("i" + i + " : j " + j);
            }
        }
    }
}
```

OP: i1 : j1  
 i2 : j1  
 i3 : j1

--when we should use continue statement

- ① skipping unwanted conditions which improves code readability.
- ② Validating input and skipping them which makes cleaner loops.
- ③ In nested loops, we can use label and control specific loop.
- ④ In iterations with conditions we can avoid nested if-else block replacing them with continue.

NOTE: continue  $\rightarrow$  skips 1 iteration

break  $\rightarrow$  exit the loop/switch entirely

return  $\rightarrow$  exit the method

;  $\rightarrow$  terminate the statement

\* label.

- In java, a label is an identifier followed by a colon (:)
- It is used to name a loop or block of code.
- Label is mainly used with break and continue statements to control the flow of execution out of nested loops.

Ex: class LabelExample

```
{ main()
```

```
{ outerLoop:
```

```
for (int i = 1; i <= 10; i++)
```

```
{ if (i % 3 == 0) continue outerLoop;
```

```
    SOP(i);
```

```
    }
```

```
}
```

Ex: class LabelEx

```
{ main()
```

```
{ evenodd: {
```

```
    SOP("Enter number:");
```

## \*label for blocks

- we can use label for normal block of code.  
(not just loops)

- Allows us to mark a block and use break to exit from that label block early

- Labels in blocks are rarely used in practice but it can be helpful when we want to break out of a nested block without using loop.

Ex: class LabelInBlockEx

```
{ main()
{ SOP("main exe starts");
  EvenOddBlock: {
    SOP("Even Odd block exe starts");
    if (5%2==0) {
      SOP("Even");
      break EvenOddBlock;
    } else {
      SOP("Odd");
    }
    SOP("Even odd block exe ends");
  }
  SOP("main exe ends");
}
```

O/P: If cond true:

Main exe starts

Even odd block exe starts

Even

main exe ends

If cond is false:

main exe starts

Even odd block exe starts:

Even. odd

Even odd block exe ends

main exe ends.

- If we are using break statement outside loop or switch it will create a CTE

- But we can use a break <label> which will work for loops and blocks.

- continue <label> only works for loops not for blocks

- labelling a block is legal but rarely recommended

\* label for loops

- we can declare a block for a label if needed

# for test

# Label for "for" loop

class LoopLabelExample

{ main ()

{ outerloop: {

for (int i = 1; i <= 3; i++)

{ innerLoop:

for (int j = 1; j <= 3; j++)

{

if (j == i) continue outerloop;

cout << i + " : " + j << endl;

}

}

}

}

Output.

|                    |         |
|--------------------|---------|
| <del>i1 : j2</del> | i2 : j1 |
| <del>i1 : j3</del> | i3 : j1 |
| <del>i2 : j1</del> | i3 : j2 |
| <del>i2 : j3</del> |         |
| <del>i3 : j1</del> |         |
| <del>i3 : j2</del> |         |

Ex: class LabelLoop

{ main() {

outerLoop:

{

for(int i=1; i<=3; i++)

{

innerLoop:

{

for(int j=1; j<=3; j++)

{

if(i==j) continue innerLoop;

SOP(i + " : " + j);

}

}

}

}

}

}

Output: i1 : j2

i1 : j3

i2 : j1

i2 : j3

i3 : j1

i3 : j2

## # Label for "while" loop

- when we are declaring a label for while loop without block then it must be placed before condition

Ex: class WhileLabelEx

```
{main()
```

```
{ int i=1;
```

```
  outerLoop:
```

```
  while(i<=5)
```

```
  {
```

```
    SOP(i);
```

```
    i++;
```

```
  }
}
```

Output:

1

2

3

4

5

- If we want to place a label before initialization, then we need to make use of blocks.

Ex: class WhileLabelEx

```
{main()
```

```
{ outerLoop: {
```

```
  int i=1;
```

```
  while (i<=5)
```

```
  { SOP(i);
```

```
    i++;
```

```
  }
```

```
}
}
```

Output:

1

2

3

4

5

## # Label for "do-while" Loop.

- when we are declaring a label for do while loop without blocks then it must be placed before condition.

Ex : class LabelDoWhileLoop

```
{main()
```

```
{ int i=1;
```

```
  outerloop:
```

```
  do {
```

```
    SOP(i);
```

```
    i++;
```

```
  }while (i<=5);
```

```
}
```

output :

1

2

3

4

5

- If we want to place a label before initialization then we need to make use of blocks.

Ex : class LabelDoWhileLoop2

```
{main()
```

```
{ outerloop:
```

```
{
```

```
  int i=1;
```

```
  do {
```

```
    SOP(i);
```

```
    i++;
```

```
  }while (i<=5);
```

```
}
```

```
}
```

```
}
```

output :

1

2

3

4

5